

LaunchLens

Should you launch it? A market-intelligence copilot that fuses demand + supply into one Go / No-Go / Niche verdict.

Ruby Gunna · Agent Builder 2026 — Assignment 3 · LangGraph + SerpApi + Oxylabs

THE PROBLEM

Founders launch on gut feel — and the signals are scattered

- **Demand lives in one place** Google Trends, News, Shopping — is anyone searching, is it seasonal, what does it cost elsewhere?
- **Supply lives in another** Amazon — who already sells it, at what price, and what do reviews complain about?
- **Nobody fuses them** A trend chart alone says 'maybe'. A competitor list alone says 'crowded'. The answer is in the overlap.
- **Result** Weeks of manual research, or an expensive guess. Founders need a fast, evidence-backed verdict.

One question in, one judgement out

- **Ask in plain English** “Should I launch a \$20 meal-prep container set in the US?”
- **It gathers evidence** Pulls demand (Google) + supply (Amazon) live, in parallel, then reasons across both.
- **It returns a verdict** Go / No-Go / Niche — with a price band, positioning angle, and a confidence level.
- **It remembers** Follow-ups (“what about Canada?”) keep context across the whole conversation.

A founder conversation, end to end

- **1 · Idea** “Is a 32oz insulated steel bottle worth launching in the US under \$40?” → full fan-out + verdict.
- **2 · Price** “What price should I sell it at?” → pricing branch: Google Shopping + Amazon together.
- **3 · Pain** “What are reviewers complaining about?” → mined Amazon review complaints.
- **4 · Memory** “Now compare with Canada.” → remembers the product across turns.
- **5 · Recall** “What did we conclude earlier?” → summarization keeps long chats coherent.

A typed LangGraph state machine

- **START** → **summarize** Memory node compresses old turns once the chat grows (keeps context bounded).
- → **router** Classifies intent: demand / pricing / full / chat — conditional edges pick the path.
- → **fan-out** On 'full', three nodes (Trends · Amazon · News) run in PARALLEL and merge.
- → **agent** ⇌ **tools** LLM bound to 6 tools loops until it has enough evidence.
- → **verdict** → **END** Parses the Go / No-Go / Niche label + confidence into state.

summarize

router

fan-out ×3

agent ⇌ tools

verdict

The five required concepts, mapped to code

- **1 · Graph & state** Typed StateGraph + reducer channels — state.py 16/28, graph.py 28
- **2 · Fan-out (parallel)** Router returns a 3-node list, results merge — graph.py 49–65, nodes.py 98/104/110
- **3 · Routing** Intent → conditional edges, chat = default — router.py 65/81, graph.py 49
- **4 · Agent + tools** LLM $\rightleftharpoons$ ToolNode loop over 6 tools — nodes.py 135, graph.py 41/68
- **5 · Short-term memory** SqliteSaver checkpointer + summarization — main.py 140, nodes.py 64

THE CORE IDEA

Data fusion: one judgement, never two reports

DEMAND · Google / SerpApi

- Trends — interest + seasonality (relative index, read with care)
- News — launches, recalls, market shifts
- Shopping — cross-retailer price reality

SUPPLY · Amazon / Oxylabs

- Search — who already sells it, at what price
- Product — mined review complaints (the gaps)
- Bestsellers — how entrenched the competition is

FUSED IN ONE AGENT TURN

“Interest is rising AND the \$23 bestseller's reviews complain it leaks → a real, differentiable gap.” The agent weighs demand × supply × unit-economics together and emits Go / No-Go / Niche + price band + confidence — not a demand report next to a supply report.

Six tools, two providers, slim JSON

- **google_trends** Demand curve + related queries, slope-classified (not noise-read).
- **google_news** Recent landscape: launches, recalls, competitor moves.
- **google_shopping** Cross-retailer price range for the price band.
- **amazon_search** Live listings, prices, ratings, review counts.
- **amazon_product** Per-ASIN detail incl. mined review complaints.
- **amazon_best sellers** Category leaders → how crowded / entrenched it is.
- **Every tool returns slim JSON** A few fields, never the raw payload — keeps tokens + context bounded.

Memory that survives restarts and long chats

- **Checkpoint** SqliteSaver persists full state after every node, keyed by thread_id — quit and resume.
- **Summarization node** Once the chat grows, old plain messages fold into a rolling summary.
- **Tool history stays intact** Only Human/AI text is compressed; tool-call sequences are never broken.
- **Why it matters** Per-turn cost stays roughly flat instead of growing with conversation length.

A verdict you can trust — and challenge

- **Conditional, not absolute** “Go IF COGS < \$X and a defensible feature exists” — verdicts state their bar.
- **Confidence level** High / Medium / Low, lowered when a data source comes back empty.
- **Trend sanity** Treats Trends as a relative, seasonal index — a 5→10 move isn't 'demand doubling'.
- **Quantified complaints** Weighs how common an issue is, not one cherry-picked review.
- **Unit-economics aware** Prompts margin reasoning: COGS, marketplace fees, ad cost before a Go.

Tech stack

- **Orchestration** LangGraph typed StateGraph + checkpointing.
- **LLM** Tool-calling chat model via a swappable config factory.
- **Demand data** SerpApi — Google Trends / News / Shopping.
- **Supply data** Oxylabs — Amazon search / product / bestsellers.
- **Persistence** SQLite locally; swap SqliteSaver → PostgresSaver for prod (same interface).
- **Runs offline** MOCK_MODE serves fixtures — full graph testable with no keys.

Limitations & what's next

- **Keyword routing** Cheap and explainable; an LLM classifier would handle ambiguous phrasing better.
- **Live-schema hardening** Parsers should be defensive against provider schema drift.
- **Unit-economics module** Turn margin reasoning into a real FBA fee + COGS calculator (in README roadmap).
- **Production** Postgres checkpoint for multi-user, a small eval harness, richer streaming UI.